

Exercices Conditions, Boucles et Fonctions

1. Conditions simples

1. Écrire un programme qui vérifie si un nombre donné est supérieur à 10.
 - [Codingame](#)
2. Vérifier si un nombre donné est positif, négatif ou nul.
3. Créer un programme qui affiche "Vous êtes mineur" si l'âge est inférieur à 18.
4. Écrire un programme qui vérifie si une chaîne de caractères est vide ou non.
5. Vérifier si un nombre est pair ou impair.
6. Écrire un programme qui compare deux nombres et affiche le plus grand.
7. Vérifier si un nombre est divisible par 3 ou par 5.
8. Demander à l'utilisateur un mot de passe et vérifier s'il correspond à "admin".
9. Créer un programme qui vérifie si une lettre est une voyelle ou une consonne.
10. Vérifier si une liste est vide ou non.

2. Conditions imbriquées

11. Écrire un programme qui vérifie si un nombre est compris entre 10 et 20.
12. Demander à l'utilisateur un âge et afficher s'il est un enfant (moins de 12 ans), un adolescent (entre 12 et 18 ans) ou un adulte (plus de 18 ans).
13. Créer un programme qui vérifie si une année est bissextile ou non.
14. Vérifier si un nombre est positif et multiple de 5.
15. Écrire un programme qui vérifie si une chaîne de caractères contient la lettre 'a'.
16. Vérifier si un nombre est divisible à la fois par 2 et par 3, mais pas par 9.
17. Demander à l'utilisateur un nom d'utilisateur et un mot de passe, et vérifier si les deux sont corrects.
18. Écrire un programme qui affiche "Excellent" si la note est supérieure à 85, "Bien" si elle est entre 70 et 85, sinon afficher "Peut mieux faire".
19. Créer un programme qui vérifie si une chaîne de caractères est un palindrome.
20. Écrire un programme qui vérifie si un nombre est à la fois positif, pair et supérieur à 100.
 - [CodinGame](#)

3. Boucles simples

21. Afficher les nombres de 1 à 10 avec une boucle `for`.
 - [Jeu codingame](#)
22. Afficher les nombres pairs entre 1 et 20.
23. Utiliser une boucle pour afficher chaque élément d'une liste.

```
word_list = ['apple', 'banana', 'cherry']
```

24. Demander à l'utilisateur un nombre de 1 à 10 et afficher la table de multiplication.
25. Afficher les éléments d'une chaîne de caractères un par un.
26. Utiliser `range()` pour afficher les nombres de 0 à 100 avec un pas de 10.
27. Afficher les éléments d'une liste avec leurs indices.
28. Afficher les nombres de 10 à 1 en ordre décroissant avec une boucle `for`.
29. Créer une boucle qui calcule la somme des nombres de 1 à 50.
30. Utiliser une boucle pour afficher les 5 premières lettres de l'alphabet.

- [Jeu codingame](#)

4. Boucles `while`

31. Utiliser une boucle `while` pour afficher les nombres de 1 à 10.
32. Utiliser une boucle `while` pour calculer la somme des nombres de 1 à 100.
33. Demander à l'utilisateur un nombre et continuer à demander tant que ce nombre est négatif.
34. Créer une boucle infinie qui demande un mot de passe jusqu'à ce qu'il soit correct.
35. Utiliser `while` pour afficher les multiples de 5 entre 1 et 50.
36. Écrire une boucle qui affiche les nombres de 1 à 10, puis termine avec un message "Fin de la boucle".
37. Créer une boucle `while` qui décrémente un nombre à partir de 10 jusqu'à 0.
38. Demander un nombre à l'utilisateur et calculer sa factorielle avec une boucle `while`.
39. Afficher les nombres pairs de 2 à 200 s'ils sont divisibles par 3 avec une boucle `while`.
40. Écrire un programme qui demande à l'utilisateur de saisir des nombres positifs et de les ajouter à une liste. Le programme doit continuer à demander des nombres tant que la longueur de la liste ne dépasse pas 5.

5. Utilisation des boucles avec `break` et `continue`

41. Utiliser une boucle pour afficher les nombres de 1 à 20, mais arrêter dès que vous atteignez 10.
42. Créer une boucle qui demande un nombre et s'arrête si le nombre est 0.
43. Utiliser `continue` pour afficher uniquement les nombres impairs de 1 à 10.
44. Écrire une boucle qui affiche les nombres de 1 à 20, mais saute les multiples de 3.
45. Demander à l'utilisateur un nombre et afficher tous les nombres de 1 à ce nombre, sauf ceux divisibles par 4.
46. Créer une boucle qui affiche les éléments d'une liste mais s'arrête dès qu'elle rencontre un élément spécifique.

```
elements = ['apple', 'banana', 'cherry', 'orange', 'grape']
```

```
stop_element = 'cherry'
```

47. Utiliser `break` pour quitter une boucle infinie après 5 itérations.
48. Utiliser `continue` pour ignorer les lettres 'a' et 'e' lors de l'affichage des lettres d'un mot.
49. Écrire une boucle qui demande à l'utilisateur de saisir un mot, mais qui s'arrête dès qu'il entre "stop".
50. Afficher les éléments d'une liste, arrêter la boucle si un même élément apparaît une deuxième fois.

```
elements = ['apple', 'banana', 'cherry', 'orange', 'grape', 'apple', 'banana', 'cherry',  
'orange', 'grape']
```

6. Boucles imbriquées

51. Écrire un programme qui affiche la table de multiplication de 1 à 10 avec des boucles imbriquées.
52. Créer un programme qui affiche un rectangle de caractères "X" de taille donnée par l'utilisateur.
53. Utiliser des boucles imbriquées pour afficher une pyramide de nombres.
54. Afficher tous les éléments d'une liste de listes avec des boucles imbriquées.

```
elements = [  
  
    [5, 89, 79, 4, False],  
  
    [8, 9, 12, 'azed', 7.7, 9],  
  
    [7.12, "aba", 4689, True, 3],  
  
]
```

55. Créer une boucle qui parcourt deux listes de mêmes longueurs simultanément et affiche leurs éléments côte à côte.

```
list_0 = [5, 89, 79, 4, False]
```

```
list_1 = [8, 9, 12, 'azed', 7.7]
```

56. Utiliser des boucles imbriquées pour calculer et afficher le produit scalaire de deux vecteurs.

```
vector_0 = [5, 89, 79, 4, 56]
```

```
vector_1 = [8, 9.4, 12, 78, 7.7]
```

57. Écrire un programme qui parcourt une liste de chaînes de caractères et affiche chaque lettre individuellement.
58. Créer une boucle imbriquée pour générer les coordonnées (x, y) d'un quadrillage de 0 à 4.

59. Créer une grille de caractères où chaque ligne et colonne est remplie de l'indice correspondant.
60. Afficher les éléments en diagonale d'une matrice carrée.

7. Exercices sur les listes et dictionnaires

61. Utiliser une boucle pour inverser une liste.

```
elements = [8, 9, 'mrt', 97, True, 1.2]
```

62. Utiliser une boucle pour vérifier si une liste est triée ou non.

```
my_list = [4, 2, 9, 12, 4, 32, 13, 8, 78, 1, 4, 21, 7, 4, 1, 3, 5, 4, 6]
```

63. Créer une boucle qui compte le nombre de fois qu'un élément apparaît dans une liste.

```
my_list = [4, 2, 9, 12, 4, 32, 13, 8, 78, 1, 4, 21, 7, 4, 1, 3, 5, 4, 6]
```

64. Créer un programme qui enlève tous les éléments d'une liste qui sont inférieurs à 10.

```
my_list = [4, 2, 9, 12, 4, 32, 13, 8, 78, 1, 4, 21, 7, 4, 1, 3, 5, 4, 6]
```

65. Parcourir un dictionnaire et afficher ses clés et valeurs.

```
my_dict = {  
    'nom': 'Alice',  
    'age': 25,  
    'ville': 'Paris',  
    'profession': 'Développeuse',  
    'langages': ['Python', 'JavaScript', 'C++']  
}
```

66. Utiliser une boucle pour compter le nombre de voyelles dans une chaîne de caractères.

67. Écrire une boucle qui fusionne deux listes en une seule.

```
elements = [8, 9, 'mrt', 97, True, 1.2]
```

```
my_list = [4, 2, 9, 12, 4, 32, 13, 8, 78]
```

68. Utiliser une boucle pour trouver l'élément le plus grand dans une liste.

```
my_list = [4, 2, 9, 12, 4, 32, 13, 8, 78, 1, 4, 21, 7, 4, 1, 3, 5, 4, 6]
```

69. Créer une boucle qui copie tous les éléments d'une liste dans une autre liste.

```
first_list = [8, 9, 'mrt', 97, True, 1.2, 4, 2, 9, 12, 4, 32, 13, 8, 78]
```

70. Écrire une boucle qui trie une liste sans utiliser la méthode `.sort()`.

```
messy_list = [4, 2, 9, 12, 32, 13, 8, 78, 14, 21, 7, 1, 3, 5, 6]
```

- [Jeu CodinGame](#)

8. Problèmes mathématiques et boucles

71. Créer une boucle qui affiche tous les nombres premiers entre 1 et 100.

72. Écrire une boucle qui calcule le produit de tous les éléments d'une liste.

```
my_list = [1, 2, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35]
```

73. Utiliser une boucle pour calculer la moyenne d'une liste de nombres.

```
my_list = [1, 2, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35]
```

74. Écrire une boucle qui vérifie si un nombre est un palindrome numérique.

75. Créer une boucle qui calcule les 10 premiers termes de la suite de Fibonacci.

76. Utiliser une boucle pour vérifier si un nombre est parfait.

77. Créer une boucle qui génère et affiche les carrés des nombres de 1 à 20.

78. Écrire une boucle qui affiche tous les multiples de 7 inférieurs à 100.

79. Créer une boucle qui calcule la somme des chiffres d'un nombre donné.

80. Utiliser une boucle pour déterminer si deux listes sont identiques.

```
my_list_0 = [1, 2, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35]
```

```
my_list_1 = [1, 2, 3, 5, 7, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 9, 11, 13, 15]
```

- [Jeu CodinGame](#)

9. Exercices avancés

81. Créer un programme qui vérifie si deux chaînes sont des anagrammes.

82. Écrire une boucle qui parcourt un texte et compte le nombre de mots.

83. Créer une boucle qui parcourt une liste de listes et affiche la somme de chaque sous-liste.

```
elements = [
```

```
    [5, 89, 79, 4, 12],
```

```
    [8, 9, 12, 52, 7, 9],
```

```
[71, 4689, 1, 3],  
]
```

84. Utiliser une boucle pour trier une liste de tuples en fonction de la deuxième valeur.

```
elements = [  
    (71, 4689, 1, 3),  
    (5, 89, 79, 4, 12),  
    (8, 9, 12, 52, 7, 9),  
    (9, 1, 567, 10, 11)  
]
```

85. Créer une boucle qui simule un lancer de dés jusqu'à obtenir un 6.

86. Créer une boucle qui simule un lancer de dés et continue de demander jusqu'à ce que l'utilisateur trouve.

87. Créer un programme qui génère un nombre aléatoire entre 1 et 100 et indique plus ou moins jusqu'à ce que l'utilisateur trouve.

88. Utiliser des boucles pour créer une liste de toutes les paires possibles d'une liste donnée.

```
my_list = [5, 89, 79, 4, 12]
```

89. Créer une boucle qui recherche tous les indices d'une valeur spécifique dans une liste.

```
my_list = [5, 89, 12, 5, 89, 79, 4, 12, 5, 12, 5, 89, 79, 4, 12, 5, 89, 79, 4, 12]  
  
n = 12
```

90. Écrire une boucle qui divise une liste en plusieurs sous-listes de taille égale.

```
my_list = [5, 89, 12, 592, 9, 79, 4, 812, 15, 142, 1, 76, 5, 89, 79, 4, 12, 598, 8, 79, 4,  
128, 21]
```

- [Jeu CodinGame](#)

10. Projets plus complexes

91. Calculer la somme des nombres d'une liste donnée.

```
numbers = [1, 2, 3, 4, 5]
```

92. Trouver le plus grand et le plus petit nombre dans une liste.

```
numbers = [1, 2, 3, 4, 5]
```

93. Vérifier si une chaîne est un anagramme d'une autre.

94. Calculer la somme des diviseurs d'un nombre donné.

95. Vérifier si une liste contient des éléments en double.

```
my_list = [1, 2, 3, 4, 5, 1]
```

```
my_list_1 = [1, 2, 3, 4, 5]
```

96. Calculer le produit scalaire de deux listes de même taille.

```
list_1 = [1, 2, 3, 4, 5]
```

```
list_2 = [5, 4, 3, 2, 1]
```

97. Vérifier si une chaîne contient des caractères répétés.

98. Inverser les mots dans une phrase sans inverser les lettres des mots.

99. Calculer le factoriel d'un nombre de manière itérative.

100. Calculer la distance de Hamming entre deux chaînes de caractères.